

1 Introduction

For the third report, the NacionalnaKlasa team has focused on vehicle calibration, implementation of a state machine for autonomous decision-making, and implementation of a PI controller for improved steering stability. Localization was implemented using TCP and UDP communication, allowing real-time exchange of speed and angle data. The system visualizes this information in Pygame on a custom developed graph-based map. Additionally, a fully custom traffic sign detection model was trained using manually annotated frames extracted from onboard video data. All of these activities were carried out during an active exam period, and a couple of team members were affected by illness, which required task redistribution and careful planning to maintain project progress.

2 Planned activities

Due to the exam period, all team members remained flexible, contributing to tasks whenever possible, resulting in a collective effort to maintain continuous project progress:

✚ Konstantin Malešević & Filip Goldberger

Konstantin and Filip have previously focused on developing the state-based control system and implementing autonomous decision-making. In the next phase, they will work on expanding and refining driving states to handle more complex intersections and dynamic scenarios, while also improving vehicle calibration and steering accuracy.

✚ Minja Drakul & Iva Mančev

Minja and Iva have previously worked on localization, Pygame simulation, and traffic sign detection. Going forward, they will investigate whether the existing BFMC simulation environment can be used for localization and navigation testing, and continue integrating the Dijkstra algorithm for path planning into the communication workflow between the Raspberry Pi and the PC.

3 Status of planned activities

- Vehicle calibration performed along with implementation of a PI controller to correct left drift DONE
- Development and implementation of state-machine defining vehicle behavior DONE
- Creation of a graph based on the custom track map DONE
- Implementation of localization architecture using TCP and UDP DONE
- Implementation of movement simulation in Pygame, where a moving point represents the vehicle's real-time position DONE
- Training of a fully custom traffic sign detection model using frames extracted from onboard videos DONE
- Enabling testing and potential implementation of localization within BFMC simulation in progress
- Shortest-path calculation using the Dijkstra algorithm, integrated into Raspberry Pi-PC communication in progress

4 General status of the project

- ✚ Steering calibration was performed to correct the vehicle's slight left drift, and a PI controller was implemented to achieve more stable and accurate control.

- ✚ A state-based control system was developed to clearly define vehicle behavior under various driving conditions detected by the model. The system includes states and appropriate reactions for traffic lights, parking maneuvers, stop lines and stop signs, pedestrian crossings, and interactions with pedestrians. It also processes road signs such as one-way and priority road signs, and includes states for highway entry and exit, where the vehicle adapts its speed by accelerating when entering the highway and decelerating when exiting. In addition, a line-following state was implemented. The system is also capable of detecting roundabout scenarios, which are planned for implementation in future work, along with the addition of a state for overtaking or bypassing other vehicles.
- ✚ Localization was implemented using TCP for reliable communication and UDP where lower latency is preferred. As the vehicle moves along the track, it continuously transmits its current orientation (angle) and speed whenever a change occurs. Based on this information, the PC calculates the vehicle's coordinates and the change in its position, which are then visualized in real time within the graph-based map.
- ✚ Real-time movement simulation was implemented in Pygame, where a dynamic point represents the vehicle's live position based on coordinates calculated on the PC using the angle and speed data received from the Raspberry Pi.
- ✚ In parallel, a custom traffic sign detection model was developed using frames extracted from onboard video recordings. All annotations were manually created in Supervisely and the model was trained in YOLO format. Different numbers of epochs and training parameters were tested to identify the configuration that provides the best detection performance.
- ✚ A graph-based representation of the custom real-world track map was created, with nodes defining start and goal positions. The Dijkstra algorithm is used to compute the shortest path between them. Integration of this path-planning process into the communication workflow between the Raspberry Pi and the PC is currently in progress. As the vehicle reaches nodes along the path, the system transitions to the next node, enabling continuous movement along the planned route.
- ✚ It is important to note that the development, testing, and recording process was further complicated by the stochastic behavior of the car. While the algorithms work correctly when the car is freshly started, unpredictable issues appear as it runs longer. For example, commands such as speed updates are sometimes delayed, causing seemingly random bugs beyond our direct control.

5 Upcoming activities

In the next period, we plan to work on:

- ✚ Expansion and improvement of the state machine, including the definition of additional driving states and dynamic scenarios, such as roundabout navigation and vehicle overtaking.
- ✚ Implementation of the Dijkstra-based path-planning within the communication workflow between the Raspberry Pi and the PC.
- ✚ Further validation of the BFMC simulation environment as a reliable platform for localization and navigation.
- ✚ Calibrating the localization system to account for differences between commanded and actual vehicle speeds, preventing small errors from accumulating in trajectory visualization.